



المدرسة العليا
للأساتذة - مراكش
ÉCOLE NORMALE
SUPÉRIEURE - MARRAKECH

**Université Cadi Ayyad
École Normale Supérieure
Département d'Informatique**

Rapport de Projet de Fin d'Études
*Conception et développement d'un système de
gestion des visiteurs du campus : application web et
mobile pour l'ENS Marrakech.*

Réalisé par :

NOUHAYLA SABIQ

Encadré par :

Pr. MOHAMED LACHGUAR

Présenté devant le jury composé de :

Pr. MOHAMED LACHGUAR

Rapport présenté en vue de l'obtention du
Diplôme de fin d'études Licence

Année Universitaire : 2025–2026

Déclaration

Je soussigné(e), **NOUHAYLA SABIQ**, de l'École Normale Supérieure, Université Cadi Ayyad, Département d'Informatique, déclare que le présent rapport, y compris l'ensemble des textes, figures, tableaux, extraits de code et illustrations, constitue le fruit de mon travail personnel. Les sources externes utilisées ont fait l'objet de citations et références explicites. Je reconnais que tout manquement à cet engagement relèverait du plagiat, considéré comme une infraction académique passible de sanctions.

J'autorise la mise à disposition d'un exemplaire de ce rapport à des fins pédagogiques, au bénéfice des futurs étudiants et chercheurs, et j'accepte qu'il soit consulté dans l'intérêt général de l'Enseignement Supérieur et de la Recherche.

NOUHAYLA SABIQ

29 avril 2026

Remerciements

Je tiens à exprimer ma profonde gratitude à Monsieur Lachguar pour l'encadrement rigoureux et bienveillant dont il m'a fait bénéficier tout au long de la réalisation de ce projet.

Je lui suis particulièrement reconnaissante pour sa disponibilité, son écoute attentive ainsi que pour la pertinence de ses conseils et orientations, qui ont grandement contribué à la qualité de ce travail.

Son soutien constant, sa patience et ses remarques constructives m'ont permis de progresser et de mener ce projet à son terme dans les meilleures conditions.

Qu'il trouve ici l'expression de ma sincère reconnaissance et de mon profond respect.

Nouhayla Sabiq

Abstract

This project aims to design and develop a web and mobile visitor management system for the campus of the École Normale Supérieure of Marrakech. The main objective is to digitize the visitor request process, validation, entry control and exit confirmation, while ensuring full traceability of visitor movements.

The CampusVisitor system is based on a client-server architecture composed of a REST API developed with Node.js and Express.js, a web interface built with React.js, a native Android application developed in Java, and a PostgreSQL database. The project includes secure authentication using JWT, password hashing with bcrypt, and role-based access control for administrators, agents, hosts and visitors.

The administrator supervises the system and manages visitors, hosts and visits. The security agent handles access control, especially check-in. The host can view assigned visits and confirm visitor check-out. The visitor can submit visit requests and track their status through a personal dashboard.

This solution improves campus security, traceability, visit organization and communication between all actors involved in the visitor management process.

Keywords : CampusVisitor, visitor management, React.js, Node.js, Android, PostgreSQL, JWT, check-in, check-out.

Résumé

Ce projet consiste à concevoir et développer une solution web et mobile de gestion des visiteurs au sein du campus de l'École Normale Supérieure de Marrakech. L'objectif principal est de digitaliser le processus de demande de visite, de validation, de contrôle d'entrée et de confirmation de sortie, tout en assurant une traçabilité complète des mouvements des visiteurs.

Le système CampusVisitor repose sur une architecture client-serveur composée d'une API REST développée avec Node.js et Express.js, d'une interface web réalisée avec React.js, d'une application mobile Android native développée en Java, ainsi que d'une base de données PostgreSQL. Le projet intègre une authentification sécurisée par JWT, le hachage des mots de passe avec bcrypt, ainsi qu'une gestion des rôles : administrateur, agent, hôte et visiteur.

L'administrateur dispose d'un espace complet pour gérer les visiteurs, les hôtes, les demandes de visite et consulter les statistiques. L'agent assure le contrôle d'accès, notamment le check-in. L'hôte consulte ses propres visites et confirme le check-out. Le visiteur peut créer une demande de visite et suivre son état depuis son espace personnel.

Cette solution permet d'améliorer la sécurité, la traçabilité, l'organisation des visites et la communication entre les différents acteurs du campus.

Mots clés : CampusVisitor, gestion des visiteurs, React.js, Node.js, Android, PostgreSQL, JWT, check-in, check-out.

Liste des abréviations

Android	Android (Système d'exploitation mobile utilisé pour l'application mobile)
API	Application Programming Interface (Interface de Programmation d'Application)
APK	Android Package Kit (Format de fichier d'installation Android)
bcrypt	bcrypt (Algorithme de hachage utilisé pour le chiffrement des mots de passe)
CRUD	Create, Read, Update, Delete (Opérations de base sur une base de données)
CSS	Cascading Style Sheets (Feuilles de style en cascade)
DB	Database (Base de données)
HTTP	HyperText Transfer Protocol (Protocole de Transfert Hypertexte)
HTML	HyperText Markup Language (Langage de balisage hypertexte)
Java	Java (Langage de programmation utilisé pour le développement Android)
JS	JavaScript (Langage de programmation utilisé pour le frontend)
JSON	JavaScript Object Notation (Format d'échange de données)
JWT	JSON Web Token (Mécanisme d'authentification sécurisé par token)
Node.js	Node.js (Environnement d'exécution JavaScript côté serveur)
npm	Node Package Manager (Gestionnaire de paquets pour Node.js)
ORM	Object Relational Mapping (Correspondance objet-relationnel)
PostgreSQL	PostgreSQL (Système de gestion de base de données relationnelle)
RBAC	Role-Based Access Control (Contrôle d'accès basé sur les rôles)
React	React.js (Bibliothèque JavaScript pour le développement d'interfaces web)
REST	Representational State Transfer (Style d'architecture pour les API web)
SPA	Single Page Application (Application web monopage)
SQL	Structured Query Language (Langage de requête structuré)
UI	User Interface (Interface Utilisateur)
UI/UX	User Interface / User Experience (Interface Utilisateur / Expérience Utilisateur)
UML	Unified Modeling Language (Langage de Modélisation Unifié)
Volley	Volley (Bibliothèque HTTP Android pour les requêtes réseau)

Table des figures

1.1	Cycle de vie Scrum	4
1.2	Diagramme de Gantt du projet	4
2.1	Exemple de systèmes de gestion des visiteurs	8
3.1	Diagramme de cas d'utilisation du système	12
4.1	Architecture globale du système	15
4.2	Schéma de la base de données	17
4.3	Diagramme de classes	18
4.4	Diagramme de séquence : création de visite	19
4.5	Diagramme de séquence : check-in et check-out	20
5.1	Structure du backend	23
5.2	Page de connexion	23
5.3	Dashboard administrateur	24
5.4	Gestion des visites	25
5.5	Gestion des visiteurs	25
5.6	Gestion des hôtes	26
5.7	Splash screen	27
5.8	Connexion mobile	28
5.9	Interface agent	29
5.10	Interface hôte	30
5.11	Interface visiteur	31
5.12	Test login	32
5.13	Test création visite	32

Liste des tableaux

1.1	Les acteurs du système	3
2.1	Comparaison des solutions existantes	7
3.1	Les acteurs du système	9
3.2	Cas d'utilisation principaux	13
4.1	Structure simplifiée des tables	16
4.2	Endpoints API	20

Table des matières

Abstract	iii
Résumé	iv
Liste des abréviations	v
Table des figures	vi
Liste des tableaux	vii
Introduction générale	1
1 Contexte général du projet	2
1.1 Introduction	2
1.2 Présentation du projet	2
1.2.1 Fonctionnalités principales	2
1.3 Problématique	2
1.4 Objectifs du projet	3
1.4.1 Objectifs fonctionnels	3
1.4.2 Objectifs techniques	3
1.5 Acteurs du système	3
1.6 Méthodologie adoptée : Scrum	3
1.6.1 Cycle Scrum	4
1.6.2 Sprints réalisés	4
1.7 Planification du projet (Gantt)	4
1.8 Conclusion	5
2 État de l'art	6
2.1 Introduction	6
2.2 Systèmes de gestion des visiteurs	6
2.3 Solutions existantes	6
2.3.1 Systèmes basés sur des registres papier	6
2.3.2 Logiciels de gestion des visiteurs	7
2.3.3 Applications mobiles	7
2.4 Comparaison des solutions	7
2.5 Limites des solutions existantes	7
2.6 Positionnement du projet CampusVisitor	7
2.7 Illustration des systèmes existants	8
2.8 Conclusion	8

3	Analyse fonctionnelle	9
3.1	Introduction	9
3.2	Identification des acteurs	9
3.3	Besoins fonctionnels	9
3.3.1	Besoins du visiteur	9
3.3.2	Besoins de l'agent	9
3.3.3	Besoins de l'hôte	10
3.3.4	Besoins de l'administrateur	10
3.4	Besoins non fonctionnels	10
3.5	Règles métier	10
3.6	Description des scénarios	10
3.6.1	Scénario 1 : Création d'une demande	10
3.6.2	Scénario 2 : Validation	10
3.6.3	Scénario 3 : Check-in	11
3.6.4	Scénario 4 : Check-out	11
3.7	Diagramme de cas d'utilisation	12
3.8	Tableau des cas d'utilisation	13
3.9	Conclusion	13
4	Conception du système	14
4.1	Introduction	14
4.2	Architecture générale	14
4.2.1	Description de l'architecture	14
4.3	Modèle de données	15
4.3.1	Description des tables	16
4.4	Diagramme de classes	18
4.5	Diagramme de séquence	19
4.5.1	Création d'une visite	19
4.5.2	Check-in / Check-out	20
4.6	Conception de l'API	20
4.6.1	Endpoints principaux	20
4.7	Sécurité du système	21
4.8	Gestion des rôles	21
4.9	Conclusion	21
5	Réalisation du projet	22
5.1	Introduction	22
5.2	Environnement de développement	22
5.3	Réalisation du Backend	22
5.3.1	Structure du projet	22
5.3.2	Authentification	23
5.4	Réalisation de l'application Web	23
5.4.1	Page de connexion	23
5.4.2	Tableau de bord	24
5.4.3	Gestion des visites	24
5.4.4	Gestion des visiteurs	25
5.4.5	Gestion des hôtes	25
5.5	Réalisation de l'application mobile	26
5.5.1	Écran de démarrage	26
5.5.2	Connexion mobile	27
5.5.3	Interface Agent	28
5.5.4	Interface Hôte	29

5.5.5	Interface Visiteur	30
5.6	Tests avec Postman	31
5.6.1	Authentification	32
5.6.2	Création de visite	32
5.7	Résultats obtenus	32
5.8	Conclusion	33

Introduction générale

La gestion des visiteurs constitue un enjeu important pour les établissements universitaires, notamment en matière de sécurité, d'organisation et de traçabilité. Dans de nombreux campus, cette gestion repose encore sur des méthodes traditionnelles telles que les registres papier, les appels téléphoniques ou les validations manuelles. Ces pratiques peuvent entraîner un manque de visibilité sur les personnes présentes, une perte de temps et une faible traçabilité des entrées et sorties.

Dans ce contexte, le projet CampusVisitor vise à proposer une solution numérique complète permettant de gérer les demandes de visite, leur validation, ainsi que le contrôle des entrées et sorties au sein du campus de l'École Normale Supérieure de Marrakech.

La solution développée comprend une interface web destinée principalement à l'administration, ainsi qu'une application mobile Android adaptée aux différents rôles : agent, hôte et visiteur. Le système repose sur une API REST, une base de données PostgreSQL et un mécanisme d'authentification sécurisé.

Ce rapport présente les différentes étapes de conception et de réalisation du projet. Le premier chapitre expose le contexte général, la problématique, les objectifs et la méthodologie de gestion du projet. Le deuxième chapitre présente l'analyse fonctionnelle et la conception du système. Le troisième chapitre détaille l'architecture technique et les technologies utilisées. Le quatrième chapitre décrit la réalisation de l'application web, mobile et backend. Enfin, le dernier chapitre présente les tests, les résultats obtenus et les perspectives d'amélioration.

Chapitre 1

Contexte général du projet

1.1 Introduction

Dans un environnement universitaire moderne, la gestion des visiteurs constitue un enjeu important en termes de sécurité, d'organisation et de traçabilité. Les établissements tels que l'École Normale Supérieure de Marrakech accueillent quotidiennement des visiteurs (étudiants, enseignants, intervenants externes, etc.), ce qui nécessite un système efficace de gestion.

Le projet **CampusVisitor** s'inscrit dans cette optique en proposant une solution numérique permettant d'automatiser et d'optimiser le processus de gestion des visites.

1.2 Présentation du projet

CampusVisitor est une application web et mobile conçue pour gérer les visiteurs au sein du campus. Elle permet de digitaliser toutes les étapes liées à une visite, depuis la demande jusqu'à la sortie du visiteur.

Le système repose sur une architecture client-serveur et comprend :

- Une application web (React.js) pour l'administration
- Une API backend (Node.js + Express)
- Une application mobile Android pour les agents, hôtes et visiteurs
- Une base de données PostgreSQL

1.2.1 Fonctionnalités principales

- Création et gestion des demandes de visite
- Validation ou refus des visites
- Suivi en temps réel des visiteurs
- Gestion des entrées (Check-in)
- Gestion des sorties (Check-out)

1.3 Problématique

Avant la mise en place de ce système, la gestion des visiteurs était réalisée de manière manuelle, ce qui engendre plusieurs problèmes :

- Absence de traçabilité des visiteurs
- Difficulté à suivre les entrées et sorties
- Risques de sécurité élevés

- Perte de temps pour les agents
- Mauvaise organisation des rendez-vous

Face à ces limitations, il devient indispensable de mettre en place une solution numérique efficace.

1.4 Objectifs du projet

1.4.1 Objectifs fonctionnels

- Permettre aux visiteurs de créer une demande de visite
- Permettre aux administrateurs de gérer les utilisateurs
- Permettre aux agents de contrôler les entrées
- Permettre aux hôtes de valider les sorties
- Fournir un suivi complet des visites

1.4.2 Objectifs techniques

- Développer une API REST sécurisée
- Mettre en place une authentification basée sur JWT
- Créer une interface utilisateur intuitive
- Assurer la communication entre mobile et backend

1.5 Acteurs du système

Acteur	Rôle	Description
Admin	Gestion	Gère les utilisateurs et supervise le système
Agent	Contrôle	Effectue le check-in des visiteurs
Hôte	Accueil	Reçoit les visiteurs et fait le check-out
Visiteur	Demande	Crée des demandes de visite

Table 1.1 : Les acteurs du système

1.6 Méthodologie adoptée : Scrum

Pour la réalisation de ce projet, nous avons adopté la méthodologie agile Scrum. Scrum permet :

- Une meilleure organisation du travail
- Une adaptation aux changements
- Une livraison progressive du produit

1.6.1 Cycle Scrum

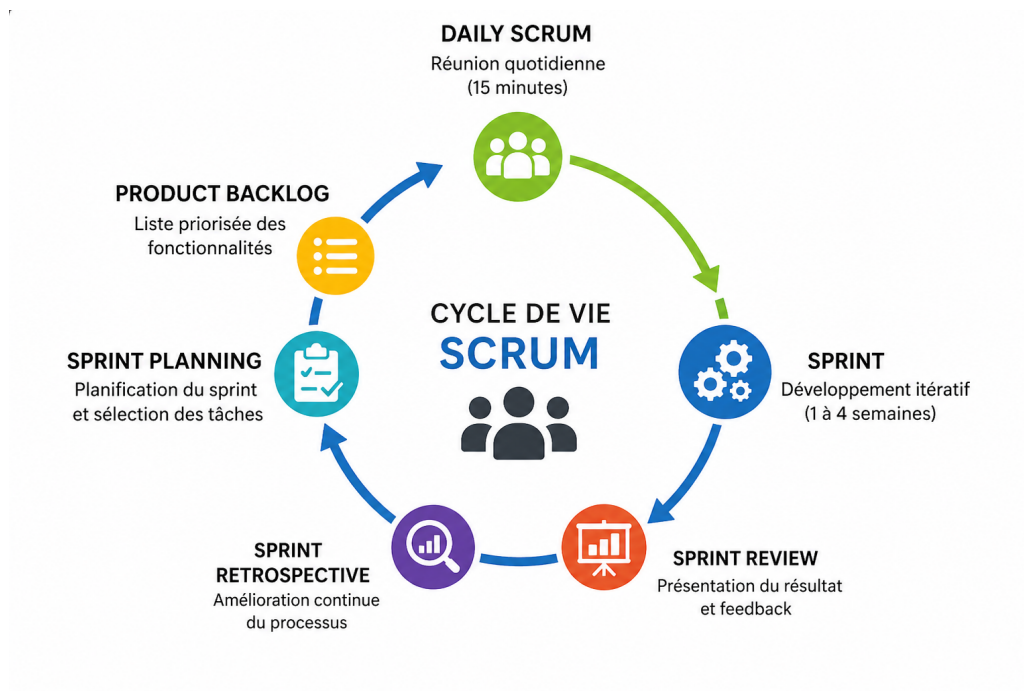


Figure 1.1 : Cycle de vie Scrum

1.6.2 Sprints réalisés

Le projet a été organisé en plusieurs sprints :

- Sprint 1 : Analyse et conception
- Sprint 2 : Développement backend
- Sprint 3 : Développement frontend
- Sprint 4 : Application mobile

1.7 Planification du projet (Gantt)

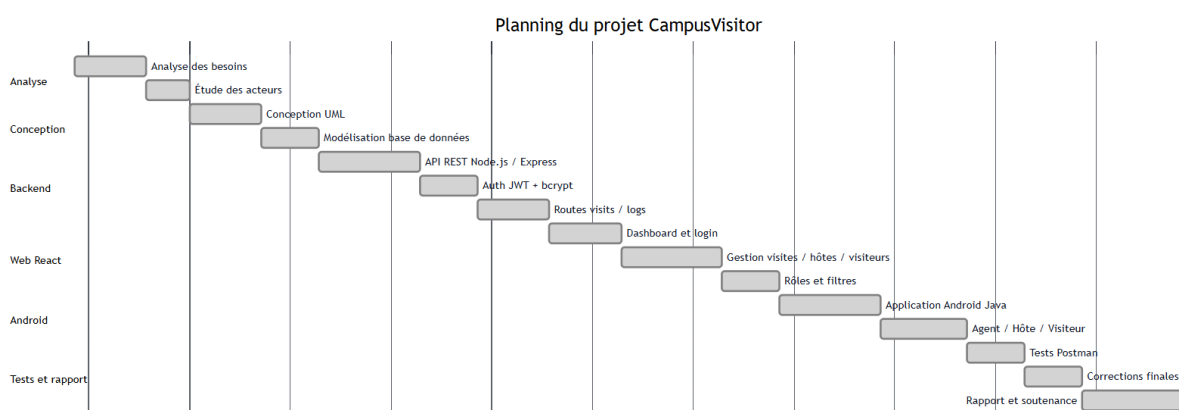


Figure 1.2 : Diagramme de Gantt du projet

1.8 Conclusion

Dans ce chapitre, nous avons présenté le contexte du projet CampusVisitor, ainsi que la problématique, les objectifs et la méthodologie adoptée. Cette étude constitue une base essentielle pour la suite du projet.

Chapitre 2

État de l'art

2.1 Introduction

Dans ce chapitre, nous présentons une étude des solutions existantes dans le domaine de la gestion des visiteurs. Cette analyse permet de mieux comprendre les systèmes actuels, leurs fonctionnalités, ainsi que leurs limites, afin de justifier le développement de la solution CampusVisitor.

2.2 Systèmes de gestion des visiteurs

Les systèmes de gestion des visiteurs sont des solutions logicielles permettant de contrôler l'accès aux bâtiments et d'assurer la traçabilité des personnes externes.

Ces systèmes offrent généralement les fonctionnalités suivantes :

- Enregistrement des visiteurs
- Attribution d'un badge ou d'un identifiant
- Suivi des entrées et sorties
- Historique des visites
- Gestion des accès sécurisés

Avec l'évolution des technologies, ces systèmes sont devenus numériques et accessibles via des applications web et mobiles.

2.3 Solutions existantes

Plusieurs solutions existent sur le marché pour la gestion des visiteurs. Parmi elles, on peut citer :

2.3.1 Systèmes basés sur des registres papier

Ce sont les systèmes traditionnels utilisés dans de nombreux établissements.

Avantages :

- Facilité d'utilisation
- Aucun coût technologique

Inconvénients :

- Manque de sécurité
- Absence de traçabilité fiable
- Difficulté de recherche

2.3.2 Logiciels de gestion des visiteurs

Certaines entreprises proposent des logiciels spécialisés permettant de gérer les visiteurs.

Fonctionnalités :

- Enregistrement numérique
- Impression de badges
- Notifications aux hôtes

2.3.3 Applications mobiles

Les applications mobiles permettent une gestion plus flexible et en temps réel.

Avantages :

- Accessibilité
- Rapidité
- Suivi en temps réel

2.4 Comparaison des solutions

Solution	Sécurité	Traçabilité	Coût
Papier	Faible	Faible	Faible
Logiciel	Moyenne	Bonne	Élevé
Mobile	Élevée	Élevée	Moyen

Table 2.1 : Comparaison des solutions existantes

2.5 Limites des solutions existantes

Malgré les avancées technologiques, les solutions actuelles présentent certaines limites :

- Coût élevé pour certaines solutions professionnelles
- Complexité d'utilisation
- Manque d'adaptation aux besoins spécifiques des universités
- Absence d'intégration entre web et mobile

2.6 Positionnement du projet CampusVisitor

Le projet CampusVisitor vise à proposer une solution complète, simple et adaptée aux besoins spécifiques du campus ENS Marrakech.

Il se distingue par :

- Une application web pour l'administration
- Une application mobile pour les utilisateurs
- Une gestion complète des rôles (admin, agent, hôte, visiteur)
- Une traçabilité en temps réel

2.7 Illustration des systèmes existants

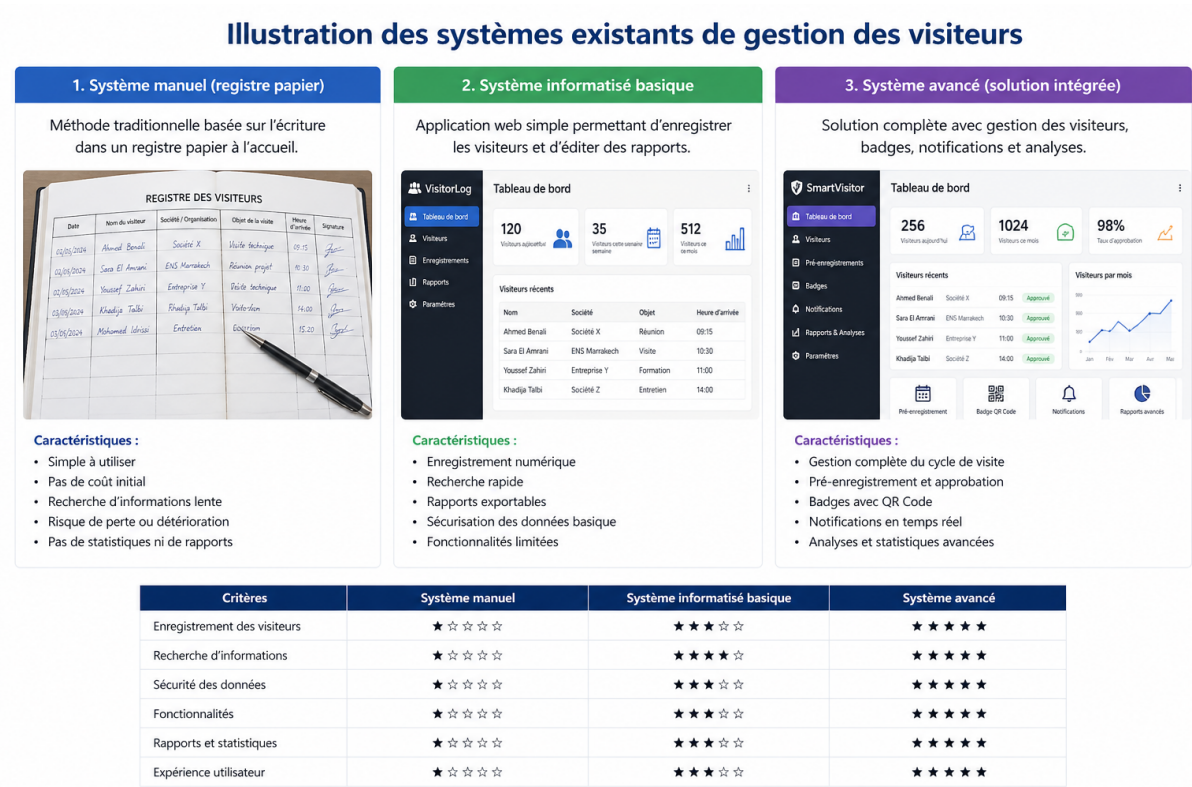


Figure 2.1 : Exemple de systèmes de gestion des visiteurs

2.8 Conclusion

Dans ce chapitre, nous avons analysé les différentes solutions existantes pour la gestion des visiteurs. Cette étude nous a permis d'identifier les limites de ces systèmes et de proposer une solution adaptée à travers le projet CampusVisitor.

Chapitre 3

Analyse fonctionnelle

3.1 Introduction

Ce chapitre présente l'analyse fonctionnelle du système CampusVisitor. Il permet d'identifier les besoins des utilisateurs, les fonctionnalités du système, ainsi que les règles métier à respecter. Cette analyse constitue une étape essentielle avant la phase de conception.

3.2 Identification des acteurs

Le système CampusVisitor met en interaction plusieurs acteurs ayant chacun un rôle spécifique.

Acteur	Rôle	Description
Administrateur	Gestion	Gère les utilisateurs et supervise le système
Agent	Sécurité	Effectue le check-in des visiteurs
Hôte	Responsable	Accueille les visiteurs et fait le check-out
Visiteur	Utilisateur	Demande une visite et suit son état

Table 3.1 : Les acteurs du système

3.3 Besoins fonctionnels

3.3.1 Besoins du visiteur

Le visiteur est l'acteur principal du système. Il doit pouvoir :

- Créer une demande de visite
- Choisir un hôte
- Définir une date et une heure
- Ajouter un objet et des notes
- Consulter l'état de sa demande (en attente, approuvée, refusée)
- Consulter l'historique de ses visites

3.3.2 Besoins de l'agent

L'agent est chargé du contrôle d'accès. Il doit pouvoir :

- Consulter les visites approuvées
- Vérifier les informations du visiteur
- Effectuer le check-in
- Consulter les visiteurs présents

3.3.3 Besoins de l'hôte

L'hôte est responsable du visiteur. Il doit pouvoir :

- Consulter ses visites
- Voir les visiteurs attendus
- Effectuer le check-out
- Consulter les informations des visiteurs

3.3.4 Besoins de l'administrateur

L'administrateur possède un accès complet au système :

- Gérer les visiteurs
- Gérer les hôtes
- Gérer les visites
- Consulter les statistiques
- Accéder aux logs

3.4 Besoins non fonctionnels

Les besoins non fonctionnels concernent les exigences techniques du système.

- **Sécurité** : authentification via JWT, mots de passe sécurisés avec bcrypt
- **Performance** : réponse rapide de l'application
- **Disponibilité** : accès permanent au système
- **Ergonomie** : interface simple et intuitive
- **Compatibilité** : support web et mobile

3.5 Règles métier

Les règles métier définissent le fonctionnement du système :

- Une visite doit obligatoirement être liée à un hôte
- Une visite commence par une demande (status = pending)
- Une visite doit être validée avant le check-in
- Le check-in est effectué uniquement par l'agent
- Le check-out est effectué par l'hôte
- Une visite terminée ne peut pas être modifiée
- Chaque action est enregistrée dans les logs

3.6 Description des scénarios

3.6.1 Scénario 1 : Création d'une demande

Le visiteur se connecte à l'application, remplit le formulaire et soumet une demande de visite.

3.6.2 Scénario 2 : Validation

L'administrateur ou l'hôte consulte la demande et décide de l'accepter ou la refuser.

3.6.3 Scénario 3 : Check-in

À l'arrivée du visiteur, l'agent vérifie les informations et valide l'entrée.

3.6.4 Scénario 4 : Check-out

À la fin de la visite, l'hôte valide la sortie du visiteur.

3.7 Diagramme de cas d'utilisation

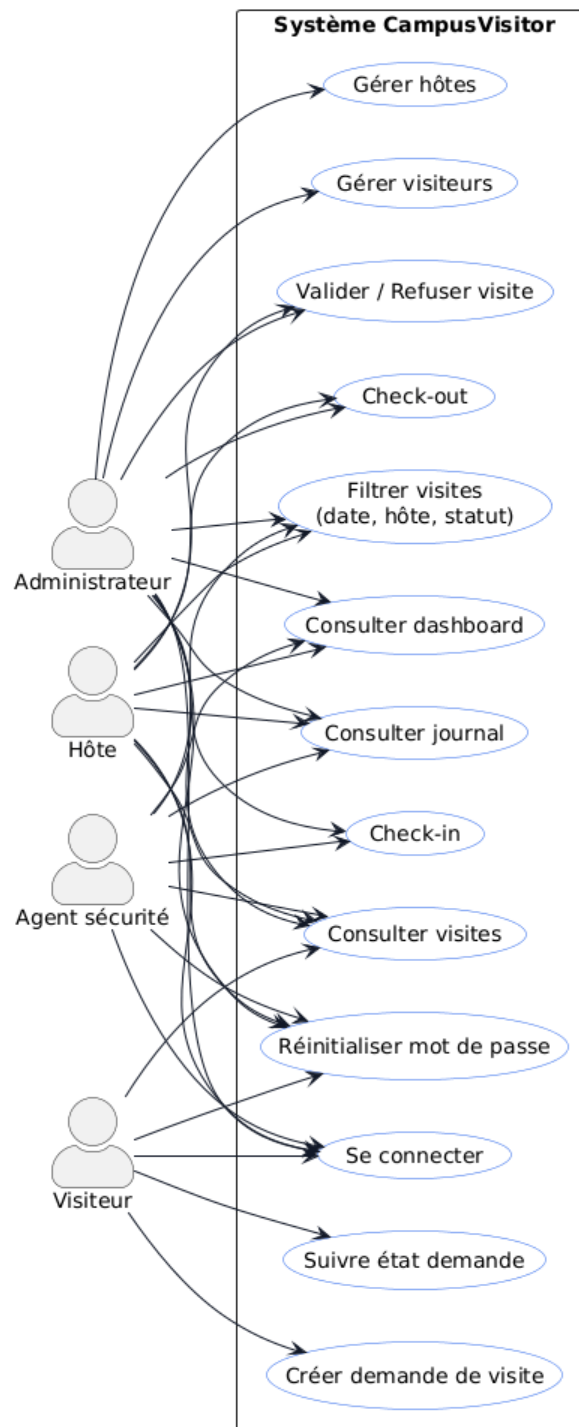


Figure 3.1 : Diagramme de cas d'utilisation du système

3.8 Tableau des cas d'utilisation

Cas	Acteur	Description
Créer visite	Visiteur	Créer une demande de visite
Valider visite	Admin	Approuver ou refuser
Check-in	Agent	Enregistrer entrée
Check-out	Hôte	Enregistrer sortie

Table 3.2 : Cas d'utilisation principaux

3.9 Conclusion

Dans ce chapitre, nous avons présenté l'analyse fonctionnelle du système CampusVisitor, en détaillant les besoins des utilisateurs, les règles métier et les scénarios d'utilisation. Cette analyse constitue une base essentielle pour la conception du système.

Chapitre 4

Conception du système

4.1 Introduction

Dans ce chapitre, nous présentons la conception technique du système CampusVisitor. Cette étape consiste à définir l'architecture globale, le modèle de données, les diagrammes UML ainsi que les interactions entre les différents composants du système.

4.2 Architecture générale

Le système CampusVisitor repose sur une architecture client-serveur composée de trois parties principales :

- **Frontend Web** : développé avec React.js
- **Backend** : développé avec Node.js et Express.js
- **Application Mobile** : développée en Java sous Android Studio
- **Base de données** : PostgreSQL

4.2.1 Description de l'architecture

- Le frontend communique avec le backend via des requêtes HTTP
- Le backend traite les requêtes et interagit avec la base de données
- L'application mobile consomme également les API REST

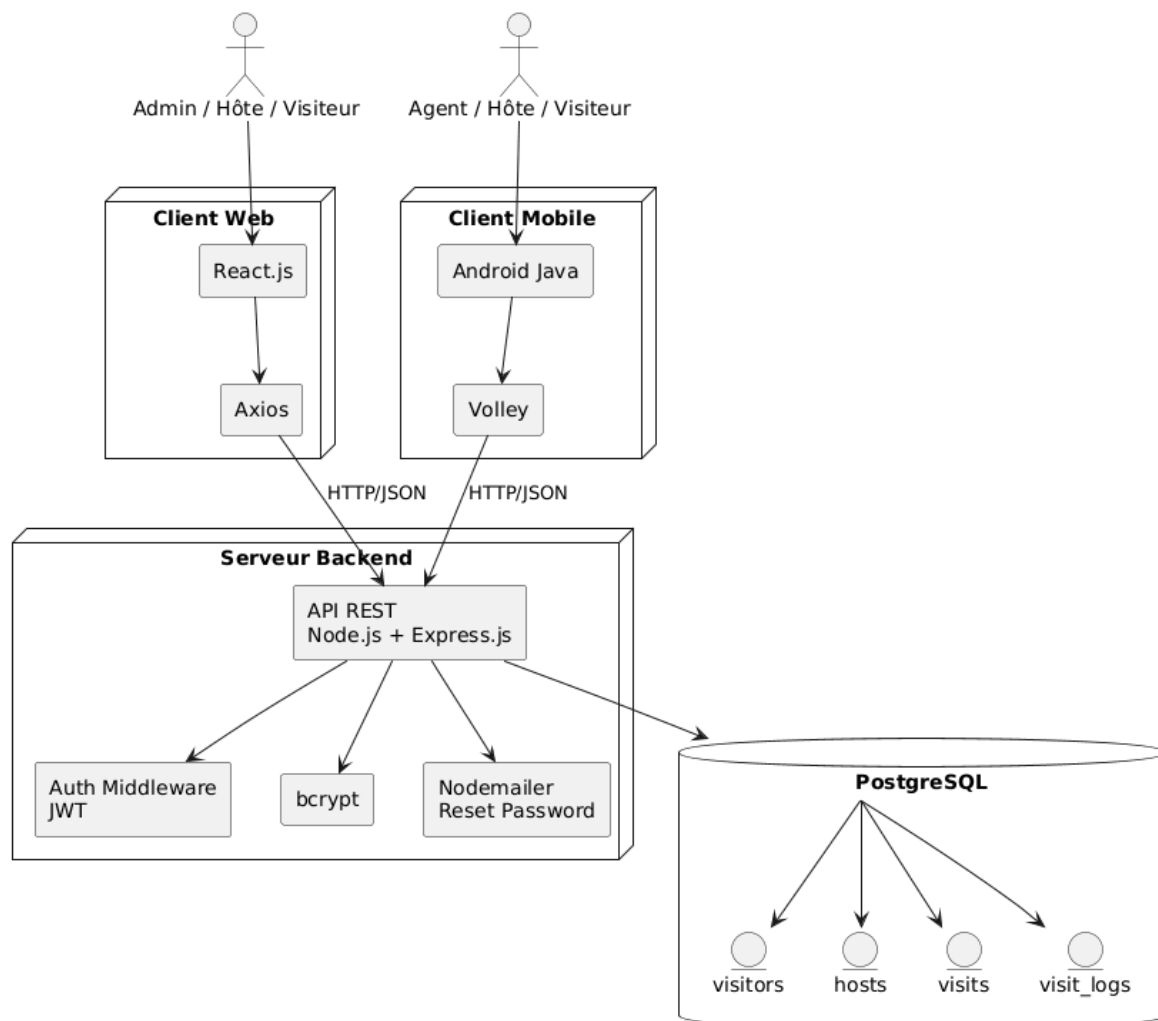


Figure 4.1 : Architecture globale du système

4.3 Modèle de données

Le système repose sur plusieurs entités principales :

- **Visitors** : informations des visiteurs
- **Hosts** : informations des hôtes et agents
- **Visits** : demandes de visite
- **Visit_logs** : historique des actions

4.3.1 Description des tables

Table	Champ	Description
Visitors	name email phone	Nom du visiteur Email Téléphone
Hosts	name department	Nom de l'hôte Département
Visits	status date	Statut de la visite Date
Visit_logs	type timestamp	Check-in / Check-out Date et heure

Table 4.1 : Structure simplifiée des tables

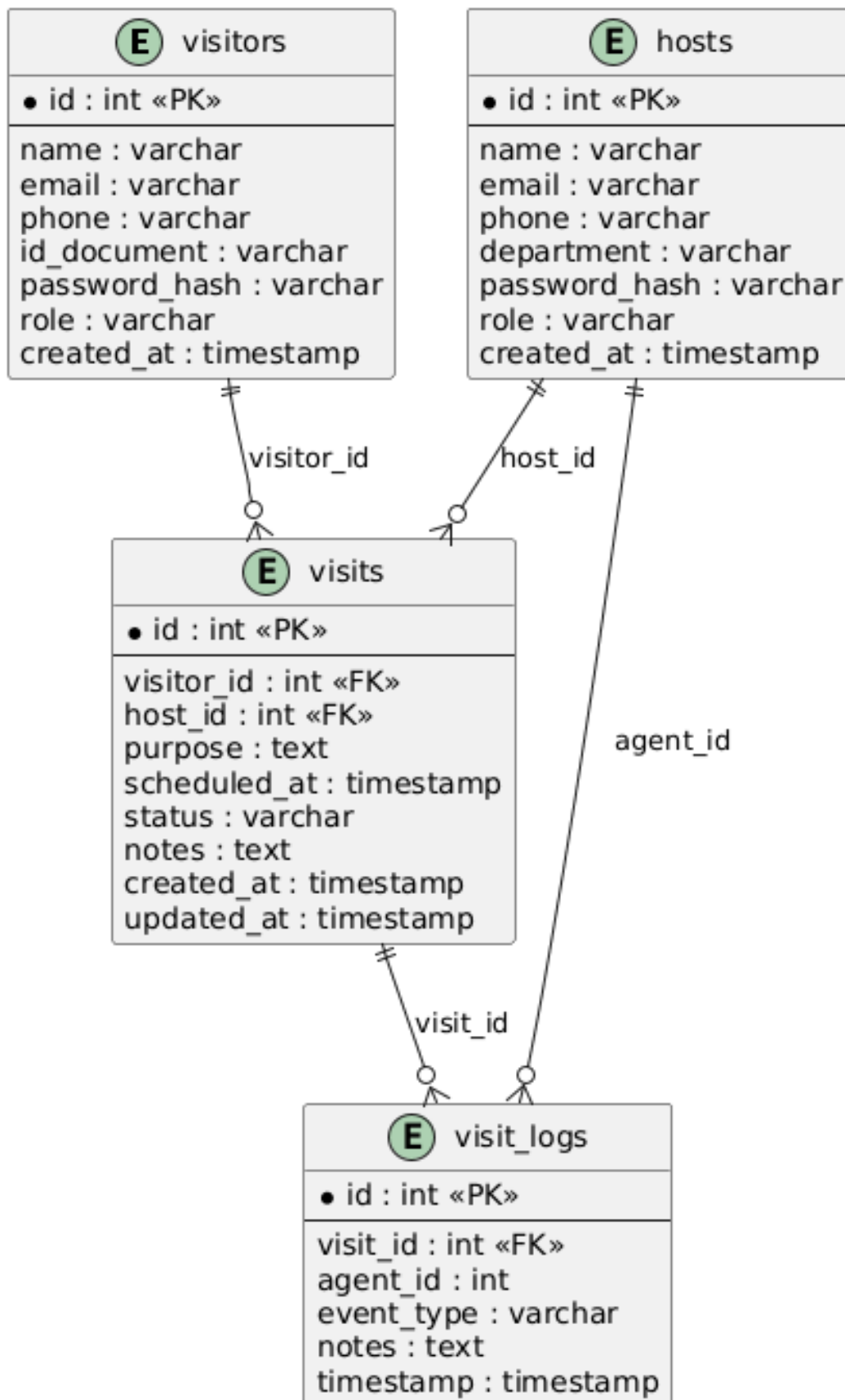


Figure 4.2 : Schéma de la base de données

4.4 Diagramme de classes

Le diagramme de classes permet de représenter les relations entre les entités du système.

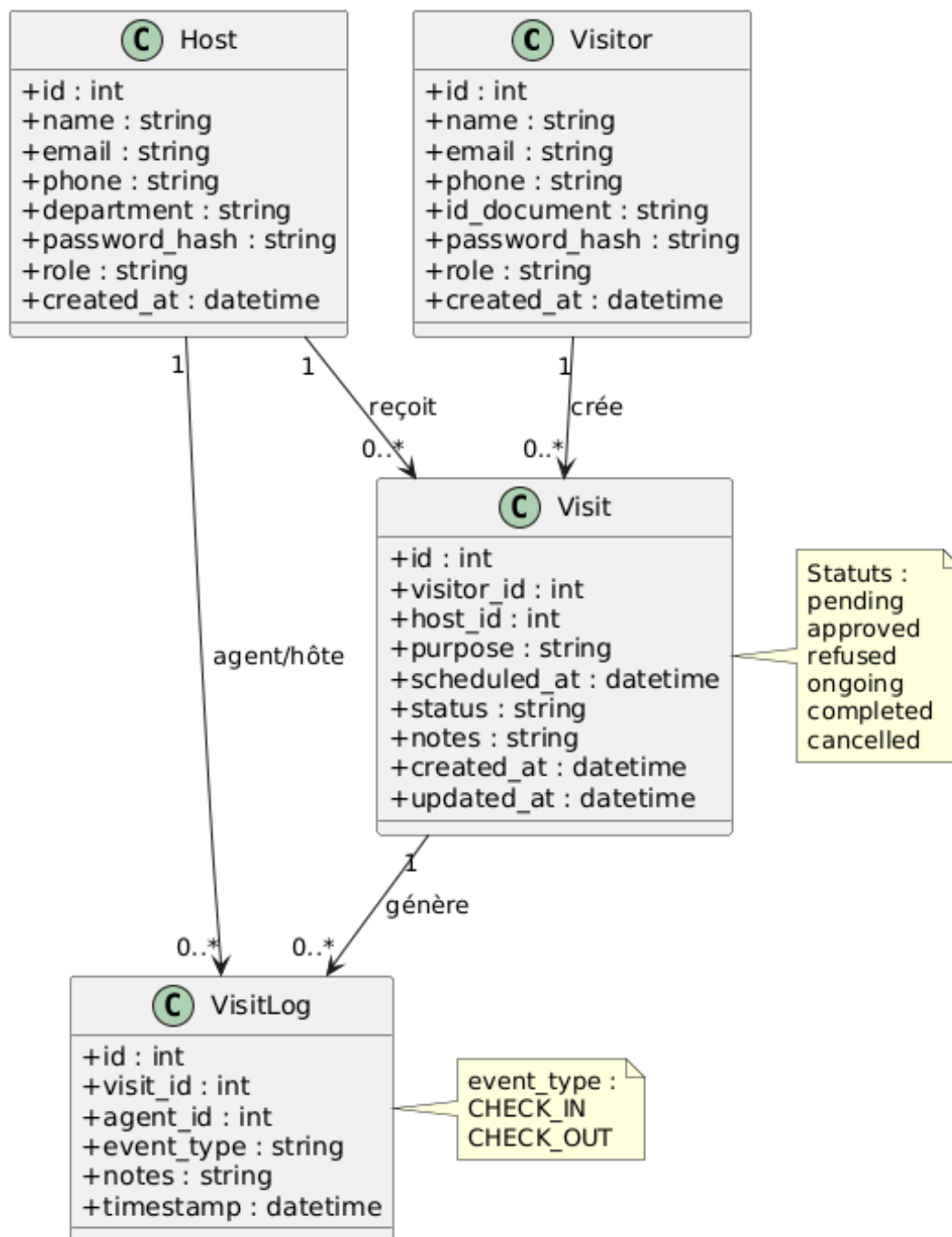


Figure 4.3 : Diagramme de classes

4.5 Diagramme de séquence

4.5.1 Création d'une visite

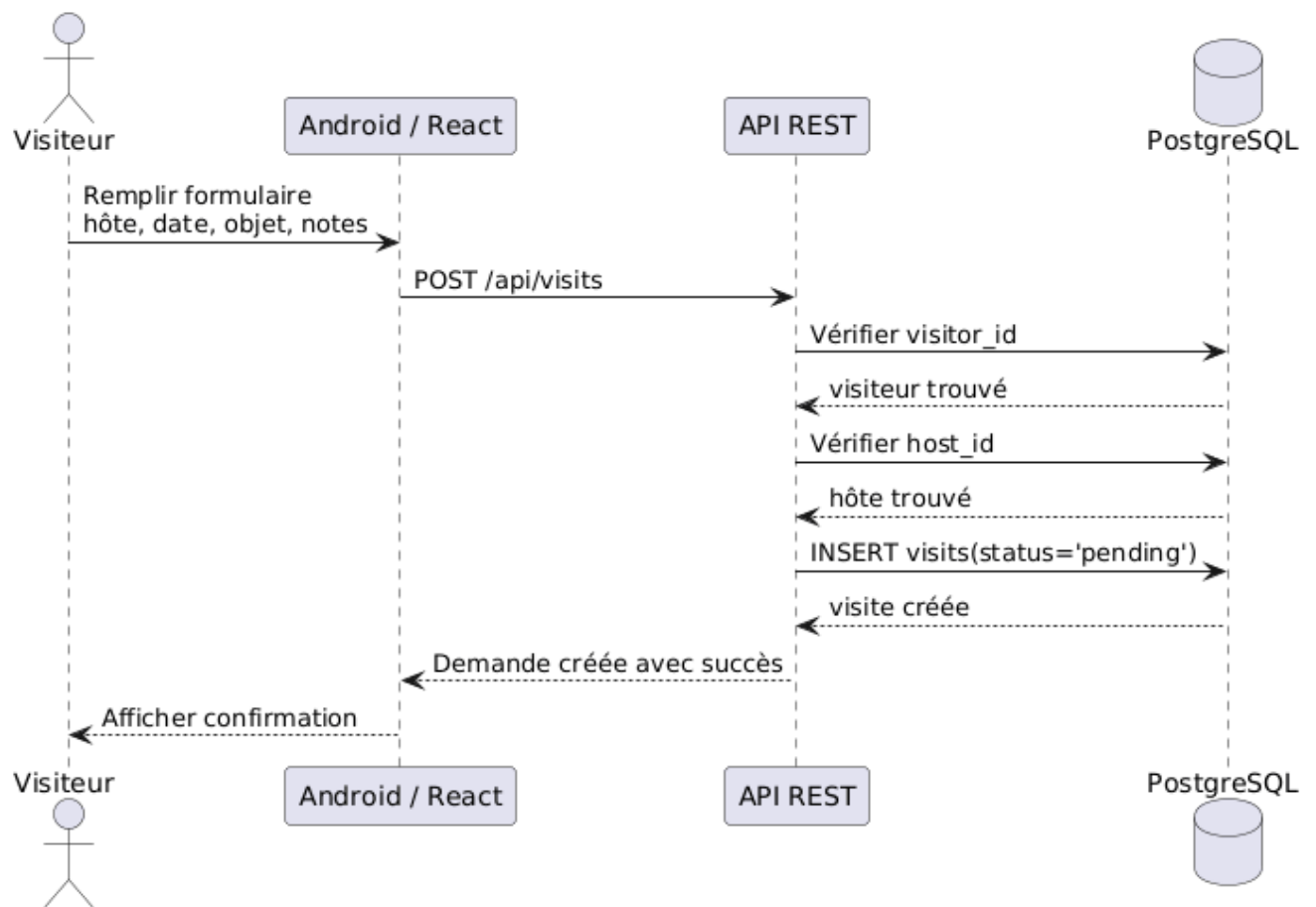


Figure 4.4 : Diagramme de séquence : création de visite

4.5.2 Check-in / Check-out

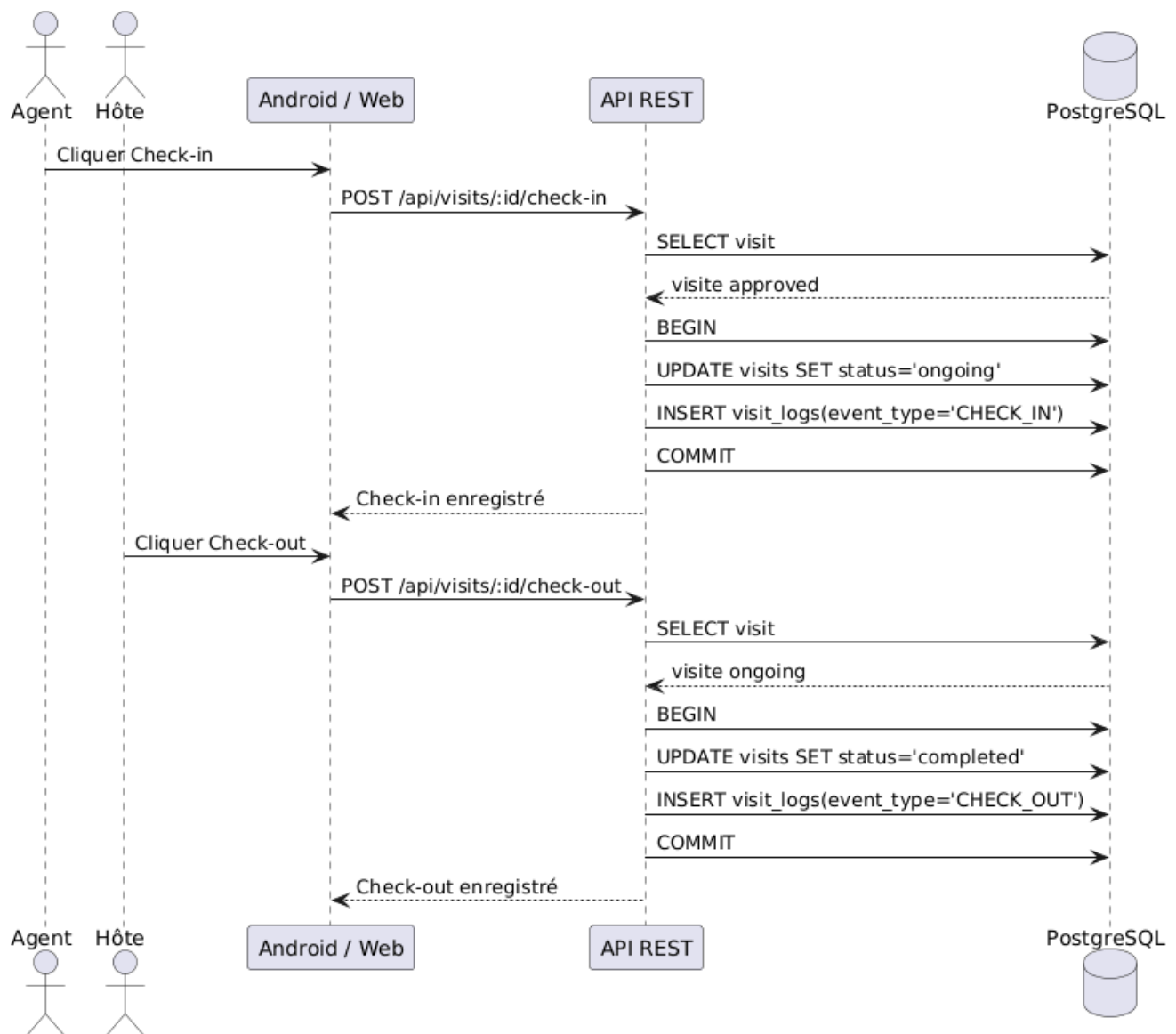


Figure 4.5 : Diagramme de séquence : check-in et check-out

4.6 Conception de l'API

L'API REST permet la communication entre les différentes parties du système.

4.6.1 Endpoints principaux

Endpoint	Méthode	Description
/api/auth/login	POST	Authentification
/api/visits	GET	Liste des visites
/api/visits	POST	Création visite
/api/visits/ :id	PUT	Modification
/api/visits/ :id	DELETE	Suppression

Table 4.2 : Endpoints API

4.7 Sécurité du système

Le système intègre plusieurs mécanismes de sécurité :

- Authentification via JWT
- Hachage des mots de passe avec bcrypt
- Gestion des rôles (Admin, Agent, Hôte, Visiteur)

4.8 Gestion des rôles

Chaque utilisateur possède un rôle qui définit ses permissions :

- **Admin** : accès complet
- **Agent** : contrôle d'accès
- **Hôte** : gestion des visites
- **Visiteur** : demande de visite

4.9 Conclusion

Dans ce chapitre, nous avons présenté la conception du système CampusVisitor en détaillant l'architecture, le modèle de données, les diagrammes UML et l'API. Cette étape constitue une base essentielle pour la réalisation du système.

Chapitre 5

Réalisation du projet

5.1 Introduction

Ce chapitre présente la phase de réalisation du projet CampusVisitor. Il décrit les différentes interfaces développées, les fonctionnalités implémentées ainsi que les outils utilisés pour la mise en œuvre du système.

5.2 Environnement de développement

Le développement du projet a été réalisé avec les outils suivants :

- Visual Studio Code pour le développement web
- Android Studio pour l'application mobile
- PostgreSQL pour la base de données
- Postman pour les tests API

5.3 Réalisation du Backend

Le backend du système a été développé en utilisant Node.js et Express.js.

5.3.1 Structure du projet

Le backend est organisé en plusieurs dossiers :

- routes
- controllers
- models
- middleware

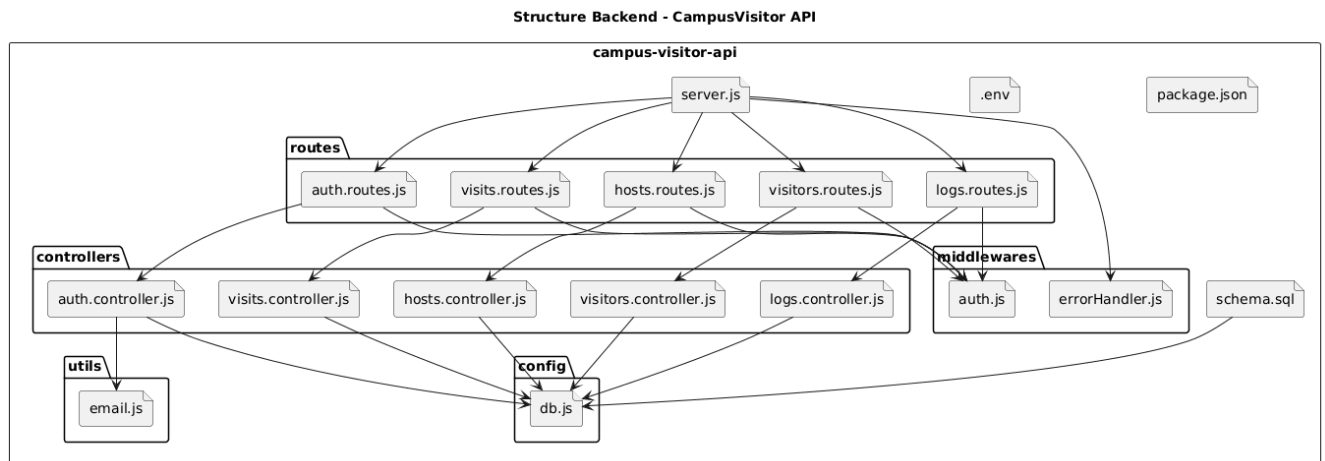


Figure 5.1 : Structure du backend

5.3.2 Authentification

Le système utilise JWT pour sécuriser l'accès.

- Connexion utilisateur
- Génération de token
- Vérification des permissions

5.4 Réalisation de l'application Web

L'interface web a été développée avec React.js.

5.4.1 Page de connexion

Cette page permet aux utilisateurs de s'authentifier.

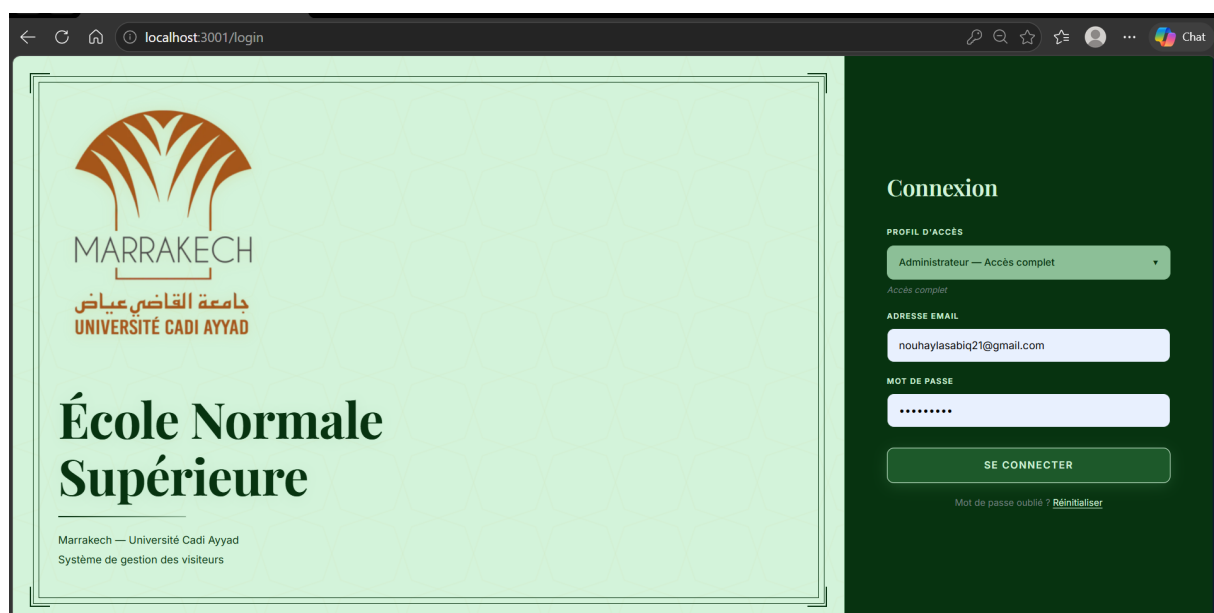


Figure 5.2 : Page de connexion

5.4.2 Tableau de bord

Le tableau de bord affiche plusieurs indicateurs :

- Nombre de visiteurs présents
- Visites du jour
- Visites refusées

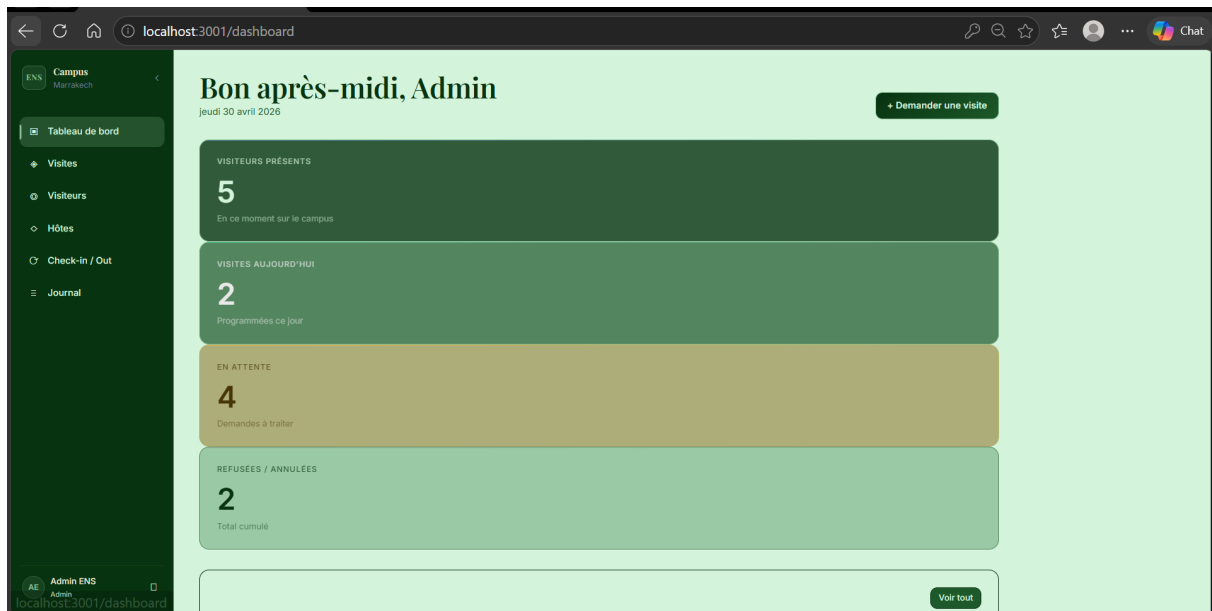


Figure 5.3 : Dashboard administrateur

5.4.3 Gestion des visites

Cette interface permet de :

- Voir toutes les visites
- Filtrer les résultats
- Valider ou refuser une demande

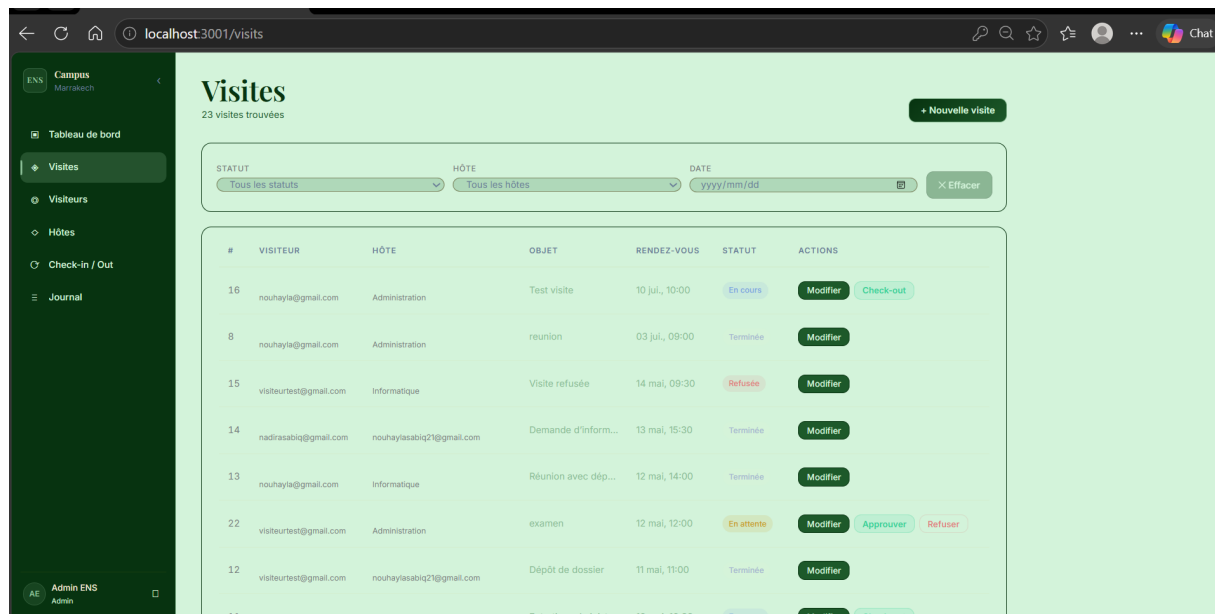


Figure 5.4 : Gestion des visites

5.4.4 Gestion des visiteurs

L'administrateur peut ajouter, modifier et supprimer des visiteurs.

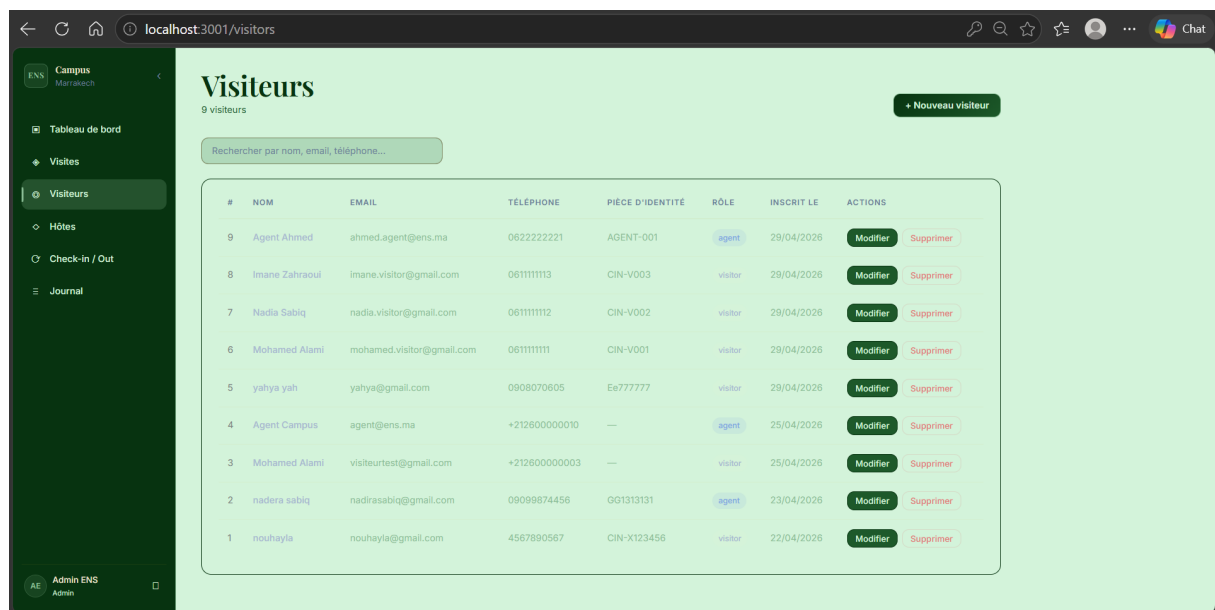


Figure 5.5 : Gestion des visiteurs

5.4.5 Gestion des hôtes

Permet de gérer les hôtes et agents.

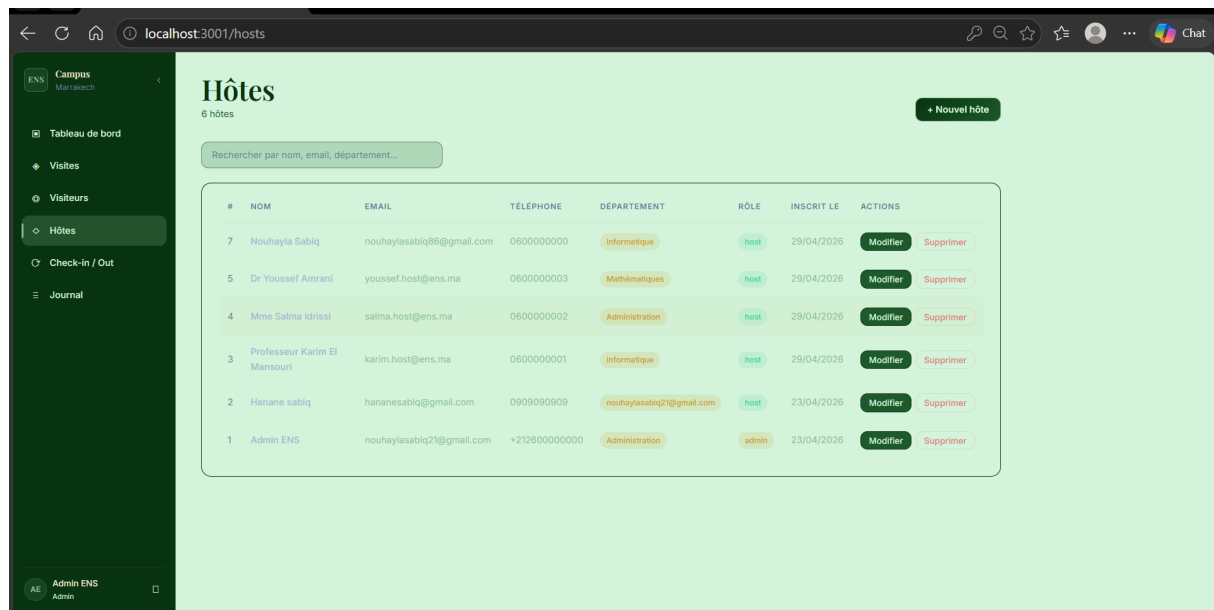


Figure 5.6 : Gestion des hôtes

5.5 Réalisation de l'application mobile

L'application Android a été développée en Java avec Android Studio.

5.5.1 Écran de démarrage

L'application contient un splash screen.



Figure 5.7 : Splash screen

5.5.2 Connexion mobile

L'utilisateur peut se connecter via email et mot de passe.



Figure 5.8 : Connexion mobile

5.5.3 Interface Agent

L'agent peut consulter les visites et effectuer le check-in.

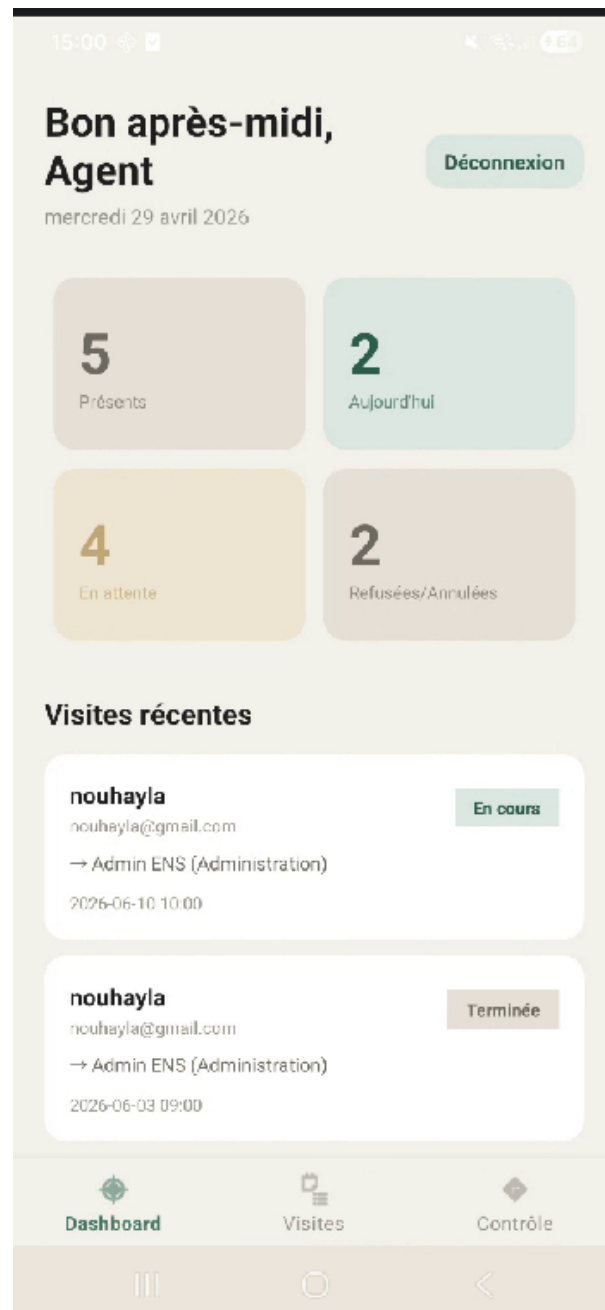


Figure 5.9 : Interface agent

5.5.4 Interface Hôte

L'hôte consulte ses visites et effectue le check-out.



Figure 5.10 : Interface hôte

5.5.5 Interface Visiteur

Le visiteur peut créer une demande et suivre son statut.

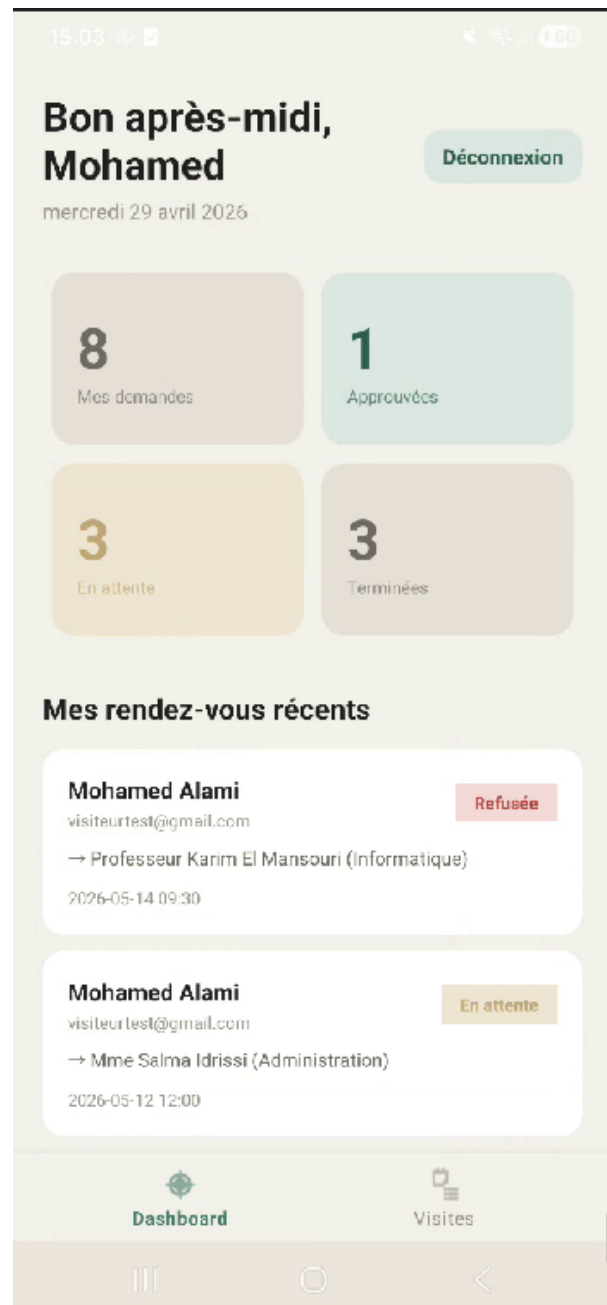


Figure 5.11 : Interface visiteur

5.6 Tests avec Postman

Les tests API ont été réalisés avec Postman.

5.6.1 Authentification

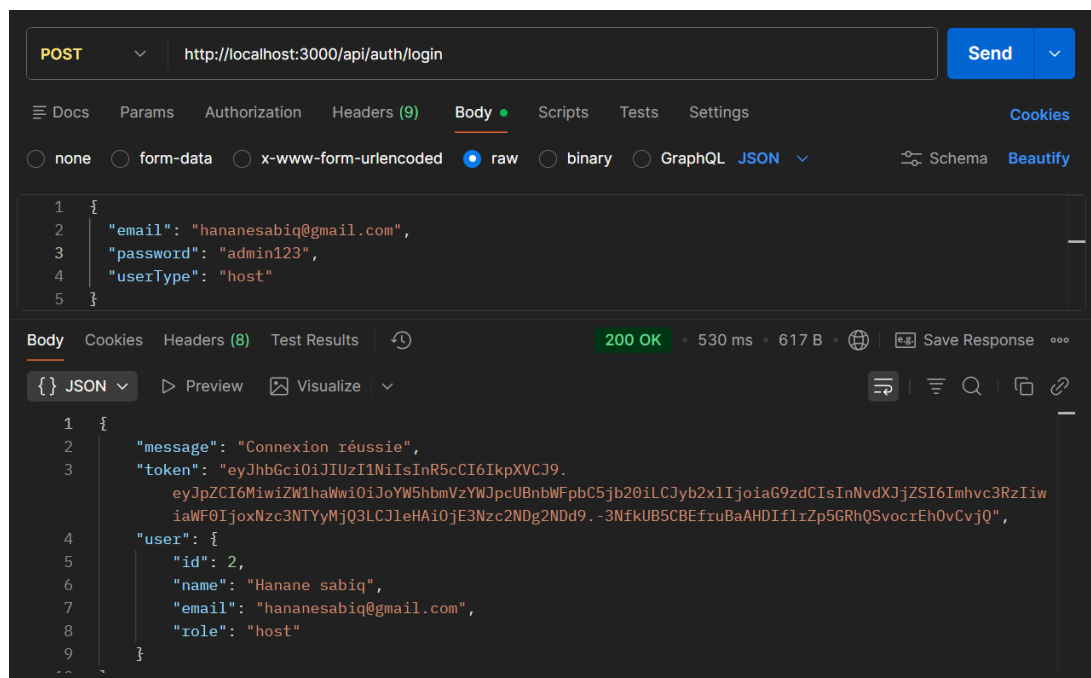


Figure 5.12 : Test login

5.6.2 Création de visite

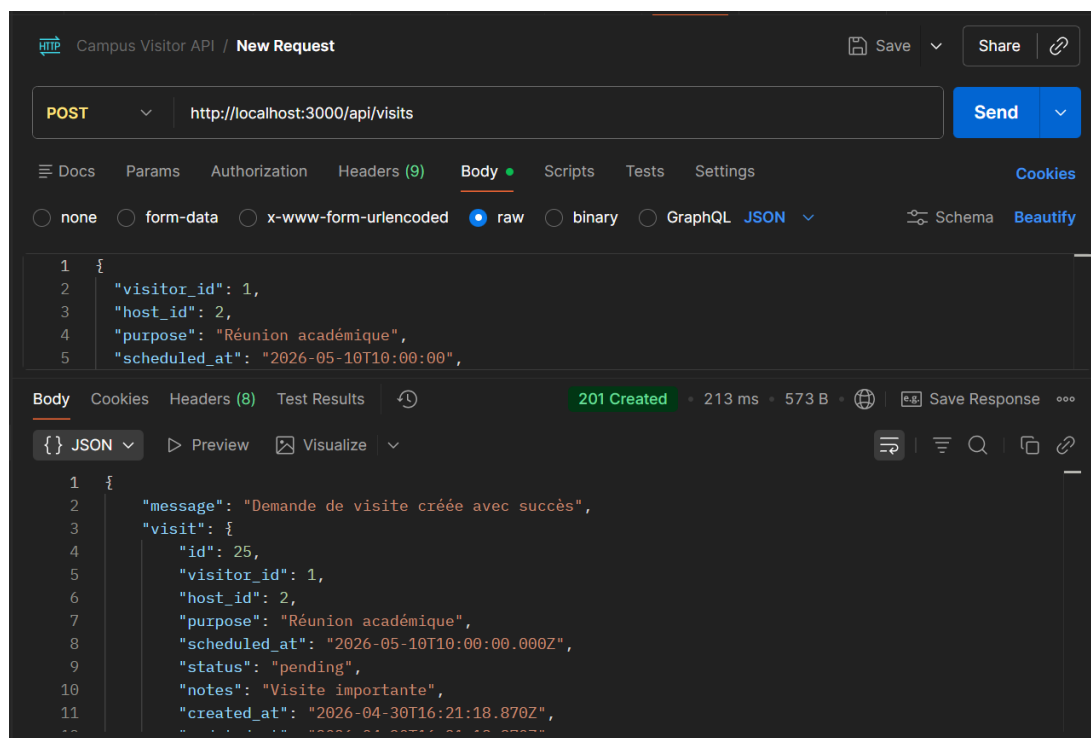


Figure 5.13 : Test création visite

5.7 Résultats obtenus

Le système fonctionne correctement et permet :

- Une gestion complète des visiteurs
- Une sécurité améliorée
- Une traçabilité des visites

5.8 Conclusion

Ce chapitre a présenté la réalisation complète du projet CampusVisitor, incluant le backend, l'application web et mobile, ainsi que les tests effectués.



NOUHAYLA SABIQ

Résumé du Projet

Ce projet consiste à concevoir et développer un système numérique de gestion des visiteurs pour le campus de l'École Normale Supérieure de Marrakech. L'objectif principal est d'améliorer la sécurité, la traçabilité et l'organisation des visites en remplaçant les méthodes manuelles par une solution automatisée.

Le système repose sur une architecture client-serveur comprenant une API REST développée avec Node.js et Express.js, une base de données PostgreSQL, une application web en React.js et une application mobile Android en Java. Il intègre un mécanisme d'authentification sécurisé basé sur JWT ainsi qu'une gestion des rôles (administrateur, agent, hôte, visiteur).

Les principales fonctionnalités incluent la gestion des demandes de visite, la validation ou le refus, le suivi des visiteurs, les opérations de check-in et check-out, ainsi que l'affichage de tableaux de bord avec statistiques. L'application mobile permet également aux utilisateurs d'interagir facilement avec le système selon leur rôle.

Ce projet a permis de développer des compétences en développement web et mobile, en conception de bases de données et en sécurité des applications, tout en apportant une solution efficace et moderne pour la gestion des visiteurs.

Mots clés : gestion des visiteurs, contrôle d'accès, API REST, React.js, Android, Node.js, PostgreSQL, JWT, authentification, campus universitaire.